

Aula02 – Banco de Dados

Professor Atila Olivi

1. Sistema Gerenciador de Banco de Dados (SGBD)

1.1. Definição e Papel Estratégico

Um **SGBD** é o cérebro que gerencia a persistência de dados. Ele não apenas guarda informações; ele garante a **Segurança, Integridade e Concorrência**. Sem ele, dois usuários tentando comprar o último ingresso de um show poderiam conseguir o mesmo assento (conflito de escrita).

1.2. Tipos de SGBD

- **Relacional (SQL):** Dados organizados em tabelas que se conectam via chaves. Foco em consistência (Ex: PostgreSQL, MySQL, SQL Server).
- **Não Relacional (NoSQL):** Focado em flexibilidade e volume. Armazena documentos, grafos ou pares chave-valor (Ex: MongoDB, Redis).

A escolha entre um banco Relacional e um Não Relacional não é sobre "qual é melhor", mas sobre "**qual é o mais adequado para o problema atual**". Enquanto o SQL foca em ordem e segurança, o NoSQL foca em velocidade e escala.

Critério	Relacional (SQL)	Não Relacional (NoSQL)
Estrutura	Baseada em Tabelas (Linhas e Colunas).	Baseada em Documentos, Grafos ou Chave-Valor.
Esquema (Schema)	Rígido: O desenho deve ser feito antes de inserir dados.	Dinâmico: Novos campos podem ser criados "na hora".
Escalabilidade	Vertical: Aumenta-se o poder do servidor (CPU/RAM).	Horizontal: Adicionam-se novos servidores (Cluster).

Critério	Relacional (SQL)	Não Relacional (NoSQL)
Integridade	Foco total nas propriedades ACID .	Foco no teorema CAP (Consistência ou Disponibilidade).
Linguagem	Padrão SQL (Select, Insert, Update).	Varia por banco (JSON-like, APIs próprias).

1.3. Análise de Vantagens e Desvantagens

Aqui analisamos o "trade-off" (troca) que o arquiteto de dados faz ao escolher cada tipo:

Tipo	Vantagens	Desvantagens
Relacional (SQL)	1. Integridade Máxima: Ideal para dados financeiros. 2. Redução de Redundância: Graças à normalização. 3. Padronização: SQL é uma linguagem universal.	1. Dificuldade de Escala: Custos altos para grandes volumes. 2. Rigidez: Mudar a estrutura de uma tabela gigante é um processo lento. 3. Performance: Pode cair em consultas com muitos JOINS.
Não Relacional (NoSQL)	1. Flexibilidade: Ideal para dados semiestruturados.	1. Consistência Eventual: O dado pode demorar milissegundos para atualizar em todos os nós.

Tipo	Vantagens	Desvantagens
	2. Alta Performance: Leitura e escrita extremamente rápidas. 3. Escala Econômica: Roda bem em hardware comum em nuvem.	2. Redundância: Dados costumam ser repetidos para ganhar velocidade. 3. Falta de Padrão: Aprender um banco não garante saber o outro.

1.4. Estrutura e Tipagem

- **Banco de Dados:** (lojas_acme)
- **Tabela:** A entidade (ex: Clientes).
- **Registro (Linha):** Uma ocorrência única (ex: O João Silva).
- **Campo (Coluna):** O atributo (ex: Email).
- **Tipos de Dados:**
 - INT: Inteiros.
 - VARCHAR(n): Texto variável.
 - DECIMAL(10,2): Valores monetários.
 - BOOLEAN: Verdadeiro/Falso.

2. Modelagem Relacional e Normalização

2.1. O Fluxo de Design

1. **Dicionário de Dados:** Documento que descreve cada campo, tipo e restrição (NOT NULL, UNIQUE).
2. **MER (Modelo Entidade Relacionamento):** Visão abstrata do negócio.
3. **DER (Diagrama Entidade Relacionamento):** O desenho técnico com Chaves Primárias (PK) e Estrangeiras (FK).

2.2. A Faxina dos Dados: As Formas Normais (FN)

Normalizar é eliminar redundâncias para evitar erros de atualização.

1ª Forma Normal (1FN): Atomicidade

Regra: Cada campo deve conter um único valor.

1. Separar telefones em linhas distintas.
2. Dividir endereço em Rua, Número e Bairro.
3. Desmembrar lista de tags de um post.
4. Acabar com campos como "Item1, Item2, Item3".
5. Garantir que não existam tabelas dentro de campos.

2ª Forma Normal (2FN): Dependência Total

Regra: Estar na 1FN e campos não-chave devem depender da PK inteira (em chaves compostas).

1. Em (ID_Pedido, ID_Produto), o Nome_Produto depende só do ID_Produto. Move-se para Produtos.
2. Em (ID_Aluno, ID_Curso), o Telefone_Aluno depende só do ID_Aluno. Move-se para Alunos.
3. Ano_Publicação depende do ISBN, não da cópia física do livro.
4. Cargo depende do Funcionário, não do Projeto onde ele atua.
5. Modelo da Máquina depende do ID_Maquina, não da Data da manutenção.

3ª Forma Normal (3FN): Dependência Transitiva

Regra: Estar na 2FN e campos não-chave não podem depender de outros campos não-chave.

1. Cidade depende do CEP, não da PK_Usuario. Move-se para Logradouros.
2. Nome_Departamento depende do ID_Departamento. Move-se para Departamentos.
3. Fabricante depende do Modelo do Carro. Move-se para Modelos.
4. Desconto_Padrão depende da Categoria. Move-se para Categorias.
5. Estádio depende do Time. Move-se para Times.

4ª Forma Normal (4FN): Dependências Multivaloradas

Regra: Separar informações independentes sobre uma mesma entidade.

1. Um Consultor fala 3 Idiomas e tem 2 Especialidades. Separar em Consultor_Idiomas e Consultor_Especialidades.
 2. Um Restaurante tem 5 Bairros de entrega e 3 Formas de Pagamento. Separar as tabelas.
 3. Um Curso tem 10 Livros e 2 Instrutores independentes. Separar.
 4. Um Produto tem várias Cores e vários Tamanhos (se a cor não restringe o tamanho).
 5. Um Usuário tem vários Hobbies e vários Dispositivos Logados.
-

3. Exemplos Contextualizados

Exemplo 1: Clínica Médica (Estrutura Básica)

Cenário: Organizar médicos e especialidades para evitar erros de digitação.

- **Tabela Especialidades:** [id_esp (PK), nome_esp] (Ex: 1, Cardiologia).
- **Tabela Medicos:** [id_med (PK), nome, crm, fk_esp (FK)].

Exemplo 2: E-commerce (N:N com Tabela Associativa)

Cenário: Um pedido tem vários produtos e vice-versa.

- **Tabela Pedidos:** [id_ped (PK), data, fk_cli].
- **Tabela Produtos:** [id_prod (PK), nome, preco].
- **Tabela Itens_Pedido:** [fk_ped, fk_prod, qtd, preco_unit]. (O preco_unit é "fotografado" aqui para histórico).

Exemplo 3: Logística (3FN)

Cenário: Evitar que o Estado seja repetido em cada entrega.

- **Tabela Cidades:** [id_cid (PK), nome_cid, uf].
 - **Tabela Entregas:** [id_ent (PK), fk_cid, endereco_destino].
-

4. Exercícios de Fixação

Nível: Fácil (Fundamentos)

1. **Tipagem:** Defina tipos de dados para uma tabela Cinema: Nome_Filme, Duracao_Min, Lancamento (Boolean), Preco_Ingresso.
2. **Chaves:** Em uma tabela Vendas, por que o CPF do cliente não deve ser a Primary Key se ele puder comprar mais de uma vez?
3. **Restrições:** Diferencie UNIQUE de PRIMARY KEY.

Nível: Médio (Relacionamentos e Modelagem)

4. **Normalização 1FN:** Transforme o campo Telefones: "99-99, 88-88" em uma estrutura normalizada.
5. **Relacionamento 1:N:** Modele um sistema de Escritorio_Contabil onde uma Empresa possui vários Contratos.
6. **Integridade:** O que acontece com os pedidos de um cliente se usarmos ON DELETE CASCADE ao excluir o cliente?
7. **Dicionário:** Crie o dicionário para a tabela Estoque (ID, Nome_Item, Qtd, Validade).
8. **Cardinalidade:** Identifique a cardinalidade entre Motorista e CNH.

Nível: Difícil (Engenharia)

9. **Chave Composta (2FN):** Tabela Curso_Aluno com PK (ID_Aluno, ID_Curso). O campo Data_Nasc_Aluno viola a 2FN? Por quê?
10. **Transitividade (3FN):** Na tabela Vendas, o campo Imposto_Estadual depende da Cidade_Destino. Como normalizar?
11. **N:N Complexo:** Modele um sistema de Streaming onde Usuarios avaliam Filmes com nota de 1 a 5 e um comentário.
12. **Dependência Multivalorada (4FN):** Um Professor orienta vários TCCs e participa de várias Bancas. Como evitar a explosão combinatória de dados?
13. **Autorrelacionamento:** Modele uma tabela Colaboradores onde um campo fk_supervisor aponte para a própria tabela.
14. **Temporalidade:** Como estruturar um banco para manter o histórico de preços de um produto ao longo dos meses?

15. **Conversão NoSQL para SQL:** Transforme um documento JSON { "pedido": 1, "tags": ["urgente", "fragil"] } em tabelas relacionais na 3FN.

5. Desafios: Situações-Problema

Desafio 1: Hospital "Vida"

Problema: O hospital mistura dados de pacientes, médicos e medicamentos em uma planilha.

Entregável: DER com 5 tabelas e Dicionário de Dados.

Resultado Esperado: Separação total entre Pacientes, Médicos, Consultas e Medicamentos (Relacionamento N:N entre Consulta e Medicamento).

Desafio 2: Fábrica de Móveis

Problema: A fábrica não sabe qual lote de madeira foi usado em qual armário, gerando perda de rastreabilidade.

Entregável: Modelo Relacional que ligue Insumos -> Lotes -> Produtos Finais.

Resultado Esperado: Garantir que um Insumo possa pertencer a vários Lotes através de uma tabela de ligação.

Desafio 3: Biblioteca Universitária

Problema: Se um livro é perdido, o sistema apaga o registro do título, perdendo o histórico de quem já leu.

Entregável: Estrutura que separe Titulo_Livro de Exemplar_Fisico.

Resultado Esperado: O aluno deve criar uma tabela Exemplares com status (Disponível, Emprestado, Extraviado) vinculada a Titulos.

6. Diferenças entre MySQL e MongoDB

Essa é a pergunta de "um milhão de dólares" (ou de algumas centenas de milhares em economia de infraestrutura). Para fechar nossa aula com chave de ouro, vamos colocar esses dois pesos-pesados no ringue.

Se o **MySQL** é um mestre de obras rigoroso que exige a planta aprovada antes de assentar o primeiro tijolo, o **MongoDB** é um artista de estilo livre que vai adaptando a obra conforme a inspiração (ou a necessidade) surge.

Aqui está o comparativo definitivo:

Característica	MySQL (Relacional)	MongoDB (NoSQL)
Modelo de Dados	Tabelas com linhas e colunas fixas.	Documentos JSON/BSON flexíveis.
Linguagem	SQL (Estruturada e padronizada).	MQL (MongoDB Query Language - baseada em objetos).
Esquema (Schema)	Rígido: Alterar uma coluna exige ALTER TABLE.	Dinâmico: Cada documento pode ter campos diferentes.
Relacionamentos	Excelente com JOINS complexos.	Foca em Embebedamento (dados aninhados) ou referências simples.
Escalabilidade	Vertical: Você precisa de um servidor "mais forte".	Horizontal: Você adiciona "mais servidores" (Sharding).
Integridade	Foco nativo em ACID (Transações pesadas).	Foco em Performance (ACID disponível, mas não é o "default" histórico).

6.1. Os 4 Pilares da Diferença

1. Estrutura de Armazenamento

No **MySQL**, você separa os dados em várias tabelas (Normalização) para não repetir informação. No **MongoDB**, você tende a agrupar tudo o que é relacionado em um único documento.

- **MySQL:** Uma tabela Clientes + Uma tabela Endereços.
- **MongoDB:** Um único documento Cliente que já contém um objeto endereço dentro dele.

2. Flexibilidade do Schema

Imagine que amanhã sua loja comece a vender sapatos e você precise do campo `tamanho_do_pe`.

- **MySQL:** Você precisa rodar um comando para criar essa coluna em todos os milhões de registros.
- **MongoDB:** Você simplesmente começa a salvar os novos documentos com esse campo. Os antigos continuam lá, sem erros.

3. Consultas (Queries)

- **MySQL:** Você usa o famoso `SELECT * FROM produtos INNER JOIN categorias...`. É uma linguagem poderosa para relatórios complexos.
- **MongoDB:** Você usa sintaxe de programação, tipo `db.produtos.find({ preco: { $gt: 100 } })`. É muito mais natural para desenvolvedores JavaScript/Python.

4. Escalabilidade (Onde o bicho pega)

Se o seu app viralizar:

- **MySQL:** Eventualmente você chegará ao limite do que uma única máquina aguenta. Escalar um banco relacional para múltiplos servidores é um desafio de engenharia complexo.
- **MongoDB:** Ele foi desenhado para o "mundo web". Ele divide os dados entre vários servidores (sharding) de forma muito mais nativa.

6.2. Quando escolher cada um?

Escolha o MySQL se:

- Você está lidando com **sistemas financeiros** ou ERPs onde a consistência é inegociável.
- Seus dados são altamente estruturados e não mudam de formato o tempo todo.
- Você precisa de relatórios complexos cruzando dezenas de tabelas.

Escolha o MongoDB se:

- Você está criando um **MVP ou Startup** onde os requisitos mudam toda semana.

- Você lida com **Big Data**, logs, ou catálogos de produtos com atributos muito variados.
- Você precisa de alta velocidade de escrita e escalabilidade horizontal simples.

Hoje em dia, a linha está ficando tênue. O MySQL já aceita campos tipo JSON e o MongoDB já suporta transações ACID complexas. A escolha agora é mais sobre a **cultura da sua equipe** e a **velocidade de evolução** do seu produto.

7.Exemplo entre MySQL e MongoDB

Vamos colocar lado a lado como o **mesmo dado** é estruturado nas duas tecnologias. Imagine um pedido de compra simples: **Um cliente, um pedido e dois itens (um mouse e um teclado).**

Abordagem MySQL (Relacional / Normalizado)

No MySQL, priorizamos a **não repetição**. Os dados são espalhados em tabelas e "colados" via Chaves Estrangeiras (FK).

Tabelas:

Cliente	
id (PK)	10
nome	Ana Silva
email	ana@email.com
fone	(11)91234-5678

Pedido	
id (PK)	500
data_pedido	17/02/2026
id_cliente (FK)	10
total	2100

Itens_Pedido			
id (PK)	1	2	3
id_pedido (FK)	500	500	500
produto	Mouse Gamer	Teclado sem fio	Monitor Gamer
qtde	1	1	1
preco_unit	100	100	2000

Como o MySQL lê isso? Ele precisa fazer um JOIN (união) das três tabelas no momento da consulta para montar o pedido completo.

Abordagem MongoDB (Documento / Agrupado)

No MongoDB, priorizamos a **leitura rápida**. O pedido é um "pacote completo" (documento) que contém tudo o que é necessário.

Coleção: pedidos

```
1  {
2    "_id": 500,
3    "data": "2026-02-17",
4    "cliente": {
5      "id": 10,
6      "nome": "Ana Silva",
7      "email": "ana@email.com"
8    },
9    "itens": [
10     { "produto": "Mouse Gamer",    "qtd": 1, "preco": 100.00 },
11     { "produto": "Teclado sem fio", "qtd": 1, "preco": 100.00 },
12     { "produto": "Monitor Gamer",  "qtd": 1, "preco": 2000.00 }
13   ],
14   "total": 2200.00
15 }
```

Como o MongoDB lê isso?

Ele faz uma única busca por `_id: 500` e já recebe o objeto pronto com os dados do cliente e a lista de itens.

Não há JOIN!